SOC L1 LAB · INC-004

Reproduction Guide — INC-004 Lab

Rebuilding cron and systemd persistence, and testing the rules against real admin work

INCIDENT	INC-004 — Cron and systemd persistence
REPORT DATE	2026-11-01
CLASSIFICATION	Lab exercise — sanitized for publication
ENVIRONMENT	Wazuh 4.14.7 · Debian 12 · QEMU/KVM
SEVERITY	Critical

Independent portfolio lab. No production systems, client data, or real credentials.

Contents

1. Prerequisites
2. Part A — Install cron (if absent)
3. Part B — Deploy the rules
4. Part C — False-positive baseline
5. Part D — Clean state
6. Part E — Run the attack
7. Part F — Verify and export
8. Part G — Cleanup
9. Troubleshooting
10. Checklist

This guide rebuilds the scenario on the existing lab. It assumes the Wazuh stack and the Debian endpoint from INC-001 are running. cron is installed during the exercise because the cloud image ships cron files but no daemon.

Time: 20–30 minutes. **Result:** two root persistence paths, four FIM alerts, a full command trail, and a false-positive baseline from legitimate cron/systemd administration.

---

1. Prerequisites

Requirement	Check
Wazuh stack	`docker compose ps`
Agent active	`agent_control -l` shows `001 soc-victim Active`
FIM realtime on `/etc`	`grep -n 'realtime.*etc' /var/ossec/etc/ossec.conf`
cron installed/active	`systemctl is-active cron`
Rules 100129/100130	`grep -c` in the container
Admin sudo	`analyst` can run `sudo -n true`

```
export LAB="$HOME/Documents/Eterna/soc-l1-lab"
export VMKEY="$LAB/lab/vm/keys/id_ed25519"
export VM="analyst@127.0.0.1"
export VMPORT=2222
```

---

2. Part A — Install cron (if absent)

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
dpkg -s cron >/dev/null 2>&1 || {
  sudo apt-get update -qq
  sudo DEBIAN_FRONTEND=noninteractive apt-get install -y -qq cron
}
systemctl is-active cron
ls -l /etc/systemd/system/multi-user.target.wants/cron.service'
```

Expected: `active` and the enable symlink. Installing cron is itself legitimate admin activity and will create a 100130 event if the rule is already deployed.

---

3. Part B — Deploy the rules

`rules/local_rules.xml` contains 100129 and 100130. The manager mounts the `rules/` directory; do not use a single-file mount.

```
cd "$HOME/wazuh-docker/single-node"
docker compose up -d wazuh.manager
sleep 20

for id in 100129 100130; do
  printf '%s: ' "$id"
  docker exec single-node-wazuh.manager-1 \
    grep -c "id=\"$id\"" /var/ossec/etc/rules/local_rules.xml
done
```

Expected: `1` for each.

Validate 100129 with a disposable cron file:

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" \
  'echo "0 3 * * * root /usr/bin/true" | sudo tee /etc/cron.d/rule-test >/dev/null'
sleep 8
docker exec single-node-wazuh.manager-1 sh -c \
  'grep "\"id\":\"100129\"" /var/ossec/logs/alerts/alerts.json | tail -1' \
  | jq -r '[.timestamp, .syscheck.path, .syscheck.event] | @tsv'
```

Expected: `/etc/cron.d/rule-test` and `added`. Remove it afterwards.

Validate 100130 with a disposable unit or an enable action:

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
sudo systemctl disable cron >/dev/null
sudo systemctl enable cron >/dev/null'
sleep 8
docker exec single-node-wazuh.manager-1 sh -c \
  'grep "\"id\":\"100130\"" /var/ossec/logs/alerts/alerts.json | tail -1' \
  | jq -r '[.timestamp, .syscheck.path, .syscheck.event] | @tsv'
```

Expected: a symlink path under `/etc/systemd/system/multi-user.target.wants/`.

---

4. Part C — False-positive baseline

This is required before the attack. Run it on an otherwise clean host.

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
echo "0 3 * * * root /usr/bin/true" | sudo tee /etc/cron.d/apt-maintenance >/dev/null
```

```
sleep 4
echo "0 4 * * * root /usr/bin/true" | sudo tee /etc/cron.d/apt-maintenance >/dev/null
sleep 4
sudo systemctl disable cron >/dev/null
sleep 3
sudo systemctl enable cron >/dev/null'
```

Expected:

- **100129** twice: the legitimate cron file added and modified.
- **100130** once: the legitimate enable symlink.
- No other 100129/100130.

Record the counts. These are the false-positive baseline for this host. Remove the test cron file:

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" 'sudo rm -f /etc/cron.d/apt-maintenance'
```

---

5. Part D — Clean state

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
ls -1 /etc/cron.d
systemctl list-timers --all | grep -i updates || true
systemctl is-active cron'
```

Expected: the stock cron files (`e2scrub_all` , `php`) and no updates timer.

Snapshot:

```
cd "$LAB/lab/vm"
./snapshot.sh save pre-inc004
```

---

6. Part E — Run the attack

Follow `04-attack-setup.md` . Summary:

```
# cron
echo "* * * * * root /usr/bin/id > /tmp/cron-proof 2>&1" \
  | sudo tee /etc/cron.d/updates

# systemd
sudo tee /etc/systemd/system/updates.service >/dev/null <<'EOF'
[Unit]
Description=Updates
[Service]
```

```
Type=oneshot
ExecStart=/bin/sh -c "id > /tmp/systemd-proof 2>&1"
EOF

sudo tee /etc/systemd/system/updates.timer >/dev/null <<'EOF'
[Unit]
Description=Updates timer
[Timer]
OnBootSec=30s
OnUnitActiveSec=60s
Unit=updates.service
[Install]
WantedBy=timers.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now updates.timer
```

Wait at least 75 seconds, then check:

```
sudo cat /tmp/cron-proof /tmp/systemd-proof
```

Expected: `uid=0(root)` twice.

---

7. Part F — Verify and export

7.1 Expected detections

```
for id in 100129 100130 80792 5402; do
  echo "== $id =="
  docker exec single-node-wazuh.manager-1 \
    grep "\"id\":\"$id\"" /var/ossec/logs/alerts/alerts.json | tail -1 \
    | jq -r '[.timestamp,.rule.id,.rule.description] | @tsv'
done
```

Expected:

```
100129  FIM: cron file added or modified
100130  FIM: systemd unit or enable-link changed
80792   Audit: Command: /usr/bin/tee
5402    Successful sudo to ROOT executed
```

7.2 Export the window

```
docker exec single-node-wazuh.manager-1 cat /var/ossec/logs/alerts/alerts.json \
  > /tmp/alerts_all.json
```

```
python3 - <<'PY'
import json, datetime
start=datetime.datetime.fromisoformat("2026-10-31T23:17:20+00:00")
end=datetime.datetime.fromisoformat("2026-10-31T23:19:30+00:00")
rows=[]
for line in open('/tmp/alerts_all.json'):
    try: a=json.loads(line)
    except: continue
    if a.get('agent',{}).get('name')=='soc-victim' and \
       start <= datetime.datetime.fromisoformat(a['timestamp']) <= end:
        rows.append(a)
json.dump(rows, open('docs/evidence/INC-004-alerts.json','w'), indent=1)
print(len(rows), "alerts")
PY
```

7.3 Preserve hashes and proofs

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
sudo sha256sum /etc/cron.d/updates \
  /etc/systemd/system/updates.service \
  /etc/systemd/system/updates.timer
sudo cat /tmp/cron-proof /tmp/systemd-proof' \
  | tee docs/evidence/INC-004-hashes-and-proof.txt
```

---

8. Part G — Cleanup

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
sudo systemctl disable --now updates.timer
sudo rm -f /etc/systemd/system/updates.service /etc/systemd/system/updates.timer
sudo systemctl daemon-reload
sudo rm -f /etc/cron.d/updates
sudo rm -f /tmp/cron-proof /tmp/systemd-proof
systemctl list-timers --all | grep -i updates || echo "no updates timer"
ls /etc/cron.d'
```

Expected: no updates timer, no `updates` cron file. cron stays installed and active.

If you took the snapshot:

```
cd "$LAB/lab/vm"
./snapshot.sh load pre-inc004
```

---

9. Troubleshooting

Symptom	Cause	Fix
`cron: inactive`	Package not installed	`apt-get install -y cron` ; `systemctl enable --now cron`
100129 does not fire	FIM not realtime on `/etc` , or parent SID wrong	Use `if_sid 550,554` ; verify `/etc` realtime
100130 fires on every enable	Symlink under `/etc/systemd/system/*.wants/`	Expected; route to change-aware triage
100130 does not fire on disable	Deletion is 553, not 550/554	Add a separate deletion rule if needed
Timer does not run	`daemon-reload` missing, or timer not enabled	`systemctl daemon-reload && systemctl enable --now updates.timer`
Cron does not run	cron inactive or file permissions wrong	`systemctl is-active cron` ; cron files must be 0644
Proof file missing	Wait less than a minute	Cron runs at the next minute; timer waits 30 s
User crontab not detected	`/var/spool/cron/crontabs` is not watched	Add FIM/audit coverage; documented gap
Large alert volume	motd/audit noise on login	Use a narrow evidence window

10. Checklist

- [ ] Agent active
- [ ] cron installed and active
- [ ] `/etc` in FIM realtime
- [ ] Rules 100129/100130 loaded
- [ ] False-positive baseline run and recorded (2× 100129, 1× 100130)
- [ ] Clean state before the attack
- [ ] Cron file and systemd units written
- [ ] Timer enabled and executed
- [ ] Both proof files show `uid=0`
- [ ] 100129 and 100130 fired
- [ ] Audit shows `tee` and `systemctl`
- [ ] Evidence exported
- [ ] Units, cron file, and proofs removed